

<Solution Q1-Open Book>

The given C code implements a singly linked list in which data of each node represents a character of a given string. Next, it removes a node if two consecutive nodes have same data. After that, reverse the list to display the modified list. Sample execution is given for easier execution of the code. Fill in the blanks with appropriate code in the EXACT LINE NUMBERS, given in the snippets.

Expected output

string is: committee

Elements in the list are:

c o m m i t t e e

Delete Duplicate Nodes

Elements in the list are:

c o m i t e

Reversed List

Elements in the list are:

e t i m o c

```
#include<stdlib.h>
#include <stdio.h>
typedef struct node *NODE;
struct node {
    char c;
    NODE next; };

NODE createNewNode(char data)
{
    NODE nu;
    nu=(NODE)malloc(sizeof(struct node));
    nu->c=data;
    nu->next=NULL;
    return nu;
}

void insertNode(NODE p, NODE k)
{
    if(k->next ==NULL) {
        k->next = p;
        p->next = NULL;
    }
    else {
        NODE t = k;
        while(t->next!=NULL)
            t = t->next;

        t->next = p;
        p->next = NULL;
    }
}
```

```

void ReverseList(NODE k)
{
    NODE karent = k->next;
    NODE prev = NULL; NODE aftr = NULL;
    while (karent != NULL) {
        aftr = karent->next;
        karent->next = prev;
        prev = karent;
        karent = aftr;
    }
    k->next = prev;
}

void removeDuplicates(NODE start)
{
    NODE karent = start;
    NODE t;

    if (karent == NULL)
        return;

    while (karent->next != NULL)
    {
        if (karent->c == karent->next->c)
        {
            t = karent->next->next;
            karent->next = t;
            free(karent->next);
        }
        else
            karent = karent->next;
    }
}

void display(NODE p) {
    NODE t = p->next;
    if (t == NULL)
        printf("List is empty.\n");
    printf("Elements in the list are: \n") ;
    while (t != NULL)
    {
        printf(" %c \t", t->c);
        t = t->next;
    }
}

int main(){
    NODE start= (NODE)malloc(sizeof(struct node));
    start->next=NULL;
    char *str="committee";

```

```

printf("string is:%s\n", str);
while(*str!='\0')
{
    NODE N=createNewNode(*str++);
    insertNode(N, start);
}
display(start);

printf("\n Delete Duplicate Nodes\n");
removeDuplicates(start);
display(start);

printf("\nReversed List\n");
ReverseList(start);
display(start);
return 0;
}

```

<Solution Q2-Open Book>

Solution:

```
#include<stdio.h>
```

```
#include <limits.h>
```

```

int findMinIndex(int arr[], int size) {
    int min = __INT_MAX__; // Initialize min value to maximum possible integer value
    int minIdx = -1; // Initialize min INDEX to unexpected integer value -1
    for (int i = 0; i < size; ++i) {
        if (arr[i] != -1 && arr[i] < min)
        {
            min = arr[i];
            minIdx = i;
        }
    }
    return minIdx;
}

```

```

int findMaxIndex(int arr[], int size) {
    int max = INT_MIN; // Initialize min to maximum possible integer value

```

```

int maxId = -1 ;
for (int i = 0; i < size; ++i) {
    if (arr[i] != -1 && arr[i] > max)
    {
        max = arr[i];
        maxId = i;
    }
}
return maxId;
}

```

/* Sorting an array with n elements in ascending order */

```

void MinMaxSort ( int arr[], int n ) {
int left = 0;           // holds the leftmost index of the working array
int right = n-1;        // holds the rightmost index of the working array
int minId, maxId, sortArr[n];

while ( left < right ) {
minId = findMinIndex( arr, n );
maxId = findMaxIndex( arr, n);
sortArr[left]=arr[minId];
sortArr[right]=arr[maxId];
arr[minId]=-1;
arr[maxId]=-1;
left = left + 1 ;
right = right - 1 ;}

printf("\nSorted array is\n");
for (int i = 0; i < n; i++)
printf("%d \t", sortArr[i]);
}

int main(){

```

```

    int arr[] = {3, 5, 112, 8, 10, 90};

```

```
int n=sizeof(arr) / sizeof(arr[0]);  
printf("%d actual array is\n", n);  
for (int i = 0; i < n; i++)  
printf("%d ", arr[i]);  
MinMaxSort(arr, n);  
}
```